

Methodological Strategy for Edge-Oriented Classification of Single-Port S_{11} Measurements

Internal Technical Report — Methodological Plan

June 2026

*This document defines the analysis pipeline before manuscript drafting. It does
not present experimental results or performance claims.*

Contents

Executive Summary	1
1 Problem Framing	2
1.1 The S_{11} Reflection Coefficient	2
1.2 Physical Origin of Class Separability	2
1.3 Classification Goal	2
1.4 Edge Deployment Context	3
1.5 Scope and Boundaries	3
2 Dataset Abstraction	4
2.1 File Format: Touchstone <code>.s1p</code>	4
2.1.1 Observed Deviation from the Standard	4
2.2 Column Interpretation	4
2.3 Sample Trace Inspection	5
2.4 Frequency Grid Validation	5
2.5 Class Distribution	5
2.6 Trace Numbering and Missing Acquisitions	6
2.7 Folder-to-Label Mapping	6
2.8 Deterministic Indexing Scheme	6
2.9 Metadata Logging Requirements	7
3 Representation Routes	7
3.1 Route A: 1D Spectral Representations	8
3.1.1 A1: Dual-Channel Cartesian Vector	8
3.1.2 A2: Flattened Cartesian Vector	8
3.1.3 A3: Derivative Augmented Cartesian Vector	8
3.1.4 Route A Encoding Summary	8
3.2 Route B: 2D Image-Like Representations	9
3.2.1 B1: Grayscale Curve Images	9
3.2.2 B2: Dual-Channel Rasterised Maps	9
3.2.3 B3: Spectrogram-Like and Heatmap Constructions	10
3.2.4 Rendering Artefact Warning	10
3.3 Route A vs. Route B: Comparative Summary	10
3.4 Representation Selection Strategy	10
4 Preprocessing Pipeline	12
4.1 Three-Stage Pipeline Overview	12
4.2 Touchstone File Parsing	12
4.3 Frequency-Grid Validation	13
4.4 Missing-Value and Malformed-File Handling	13
4.5 Outlier Inspection	14
4.6 Class-Label Mapping	14
4.7 Deterministic Sample Indexing	15
4.8 Stage 1 Summary	15
5 Normalization	15
5.1 Anti-Leakage Principle	15
5.2 Per-Feature Z-Score Normalization	16
5.3 Min-Max Scaling	16
5.4 Robust Scaling	16

5.5	Channel-Wise Normalization for Route A	17
5.6	Image Normalization for Route B	17
5.7	Comparison of Normalization Strategies	17
5.8	Practical Recommendations	18
6	Overfitting Control and Evaluation Integrity	18
6.1	Context: Small-Sample Deep Learning	18
6.2	Stratified Train / Validation / Test Split	18
6.3	Repeated Stratified k -Fold Cross-Validation	19
6.4	Nested Cross-Validation for Hyper-Parameter Tuning	19
6.5	Near-Duplicate and Temporal-Correlation Awareness	19
6.6	Capacity–Dataset Size Mismatch	20
6.7	Regularization Toolkit	20
6.7.1	Early Stopping	20
6.7.2	Weight Decay / L2 Regularization	21
6.7.3	Dropout	21
6.7.4	Label Smoothing	21
6.7.5	Pruning	21
6.8	Confidence Calibration as a Diagnostic	21
6.9	Summary of Safeguards	22
7	Data Augmentation	22
7.1	Additive Gaussian Noise	22
7.2	Frequency-Local Smooth Perturbation	23
7.3	Magnitude Scaling	23
7.4	Phase Perturbation	23
7.5	Frequency-Axis Shift	24
7.6	Mixup Augmentation	24
7.7	Image-Domain Augmentations for Route B	24
7.8	Augmentation Summary Table	25
7.9	Augmentation Budget and Ablation	25

List of Tables

1	First five and last three rows of <code>Trc0.s1p</code> (dry class) assuming RI format. Frequency is in Hz, followed by Real and Imaginary linear components.	5
2	Class distribution across the three surface-state categories. The imbalance ratio is computed relative to the minority classes.	6
3	Comparison of Route A (1D spectral) Cartesian encoding variants.	9
4	Comparison of Route A (1D spectral) and Route B (2D image-like) representation families.	11
5	Stage 1 validity-preprocessing checklist.	15
6	Comparison of normalization strategies.	17
7	Indicative parameter counts and overfitting risk relative to $N = 1603$ samples. . .	20
8	Summary of augmentation techniques for S_{11} trace data.	25

List of Figures

1	Three-stage data-conditioning pipeline. Stages 2 and 3 are covered in Sections 5 and 7, respectively.	12
---	---	----

Executive Summary

This report presents a *methodological plan* for classifying single-port S_{11} reflection-coefficient measurements acquired from an RF sensor operating in the 8–12 GHz band. The sensor is exposed to three distinct surface states—**dry**, **wet**, and **ice**—each of which alters the dielectric environment and therefore produces a characteristic frequency-dependent signature in the reflected signal. The ultimate objective is to deploy the trained classifier on resource-constrained edge hardware, where both memory and inference latency are critical constraints.

Dataset. The available corpus comprises 1603 traces distributed across three class states: **dry** (601 traces), **wet** (501 traces), and **ice** (501 traces). Each trace records 4001 uniformly spaced frequency points spanning approximately 8.0–12.0 GHz. Crucially, the dataset uses a **Cartesian representation** mapping each point to its **Real** and **Imaginary** components ($\text{Re}[S_{11}]$, $\text{Im}[S_{11}]$). This choice explicitly avoids the “phase wrapping” discontinuities inherent to polar representations (Magnitude/Phase), significantly improving numerical stability for gradient-based learning.

Dual-route representation strategy. Two complementary data representations are proposed and systematically compared throughout this report:

Route A — 1D Spectral: The Cartesian frequency-domain vectors are retained as one-dimensional feature sequences. The primary encoding is a dual-channel $\mathbb{R}^{4001 \times 2}$ vector containing the Real and Imaginary components. This route preserves full numerical fidelity and is compatible with both classical tabular models and 1D neural architectures.

Route B — 2D Image-Like: The spectral curves are rasterised into fixed-resolution images, enabling the use of established convolutional architectures designed for image classification. This route trades information fidelity for access to transfer-learning ecosystems but introduces rendering artefacts that are not present in the measurement physics.

Tiered Candidate Selection. Rather than declaring a single architecture outright, this methodology proposes a **tiered selection strategy** traversing from minimal-parameter models to deep architectures. Models are evaluated progressively, deploying the simplest model that meets accuracy requirements:

Level 0 (Unsupervised Probe): K-Means validates intrinsic data separability before supervised training.

Level 1 (Linear Baselines): Logistic Regression and Linear SVM. These are highly interpretable models whose forward pass requires mere dot products. They establish the performance floor.

Level 2 (Tabular Powerhouses): Gradient Boosted Trees (GBT), Random Forest, and RBF SVM. GBT is a standout champion here due to its consistent dominance on tabular data and invariant scaling, making it highly edge-viable without floating-point matrix arithmetic.

Level 3 (Deep Architectures): 1D CNN, MLP, ResNet-1D, and KAN. The **1D CNN** is the primary champion of this tier due to its perfect inductive alignment with local spectral resonances and extremely efficient parameter-sharing footprint.

1 Problem Framing

1.1 The S_{11} Reflection Coefficient

In microwave network analysis the scattering parameter S_{11} characterises the ratio of the complex voltage wave reflected from a one-port device to the complex voltage wave incident upon it [Pozar \(2011\)](#):

$$S_{11}(f) = \frac{V^-(f)}{V^+(f)}, \quad (1)$$

where V^+ is the incident travelling wave and V^- is the reflected travelling wave at frequency f . Because S_{11} is a complex quantity, it can be represented in either polar form (magnitude and phase) or Cartesian form (real and imaginary components). While human engineers typically prefer the polar form to visualise return loss in dB, this methodology strictly adopts the **Cartesian representation** for machine learning:

$$S_{11}(f) = \text{Re}[S_{11}(f)] + j \cdot \text{Im}[S_{11}(f)]. \quad (2)$$

The Cartesian form is mathematically superior for training gradient-based learning algorithms because it entirely eliminates the “phase wrapping” problem—the artificial discontinuity that occurs when phase angles wrap from 180° to -180° . Both the real and imaginary components are continuous, well-behaved functions of frequency, natively providing a uniform numerical scale for optimisation without requiring heuristic unwrapping algorithms.

1.2 Physical Origin of Class Separability

The sensor under study is a single-port microwave structure whose electromagnetic near-field interacts with the material present on its surface. When the surface condition changes—from a dry dielectric substrate to a thin water film or an ice layer—the effective complex permittivity $\varepsilon^* = \varepsilon' - j\varepsilon''$ of the medium shifts. This perturbation modifies the resonant frequencies, the quality factors, and the coupling coefficients of the sensor’s radiating or guided-wave structure, producing measurable changes in the $S_{11}(f)$ trace across the 8–12 GHz band.

Specifically:

- **Dry surface (dry):** The sensor is loaded only by the substrate and ambient air. The S_{11} trace exhibits the sensor’s baseline resonance profile.
- **Wet surface (wet):** A liquid water film introduces a material with high real permittivity ($\varepsilon'_{\text{water}} \approx 50\text{--}80$ at X-band frequencies) and significant dielectric loss. This shifts resonances downward in frequency, broadens them, and deepens or shallows dips depending on coupling geometry.
- **Ice-covered surface (ice):** Ice has a substantially lower permittivity ($\varepsilon'_{\text{ice}} \approx 3.2$) and much lower loss than liquid water. The resulting S_{11} perturbation is smaller in magnitude than the wet case but distinct from the dry baseline.

The classification task exploits these physically grounded spectral differences. However, the degree of separability is an empirical question that depends on the sensor geometry, the thickness and uniformity of the surface layer, and the measurement noise floor—none of which are known *a priori* from the data files alone.

1.3 Classification Goal

The formal objective is to learn a mapping

$$h : \mathbf{x}(f) \mapsto y \in \{\text{dry}, \text{wet}, \text{ice}\}, \quad (3)$$

where $\mathbf{x}(f)$ is the feature representation derived from a single S_{11} frequency sweep and y is the categorical surface-state label. This is a *closed-set, single-label, three-class* classification problem. Each trace is assumed to originate from exactly one surface condition, and no “unknown” or out-of-distribution class is modelled at this stage.

The mapping h must generalise from the 1603 available labelled traces to previously unseen measurements acquired under nominally identical conditions. The small sample size is a dominant methodological constraint: it limits the complexity of models that can be reliably validated and increases the risk of overfitting, particularly for high-dimensional deep architectures.

1.4 Edge Deployment Context

The trained classifier is intended for deployment on resource-constrained edge devices positioned at or near the sensor installation site. While the specific target hardware has not been finalised, typical edge platforms impose the following constraints:

- **Memory budget.** Model weights and inference buffers must fit within tens of kilobytes to a few megabytes of RAM, depending on the platform.
- **Compute budget.** Inference must complete within milliseconds to low hundreds of milliseconds. Floating-point units may be absent or limited to single precision; integer-only or fixed-point arithmetic may be required.
- **Power budget.** Continuous or near-continuous inference must be sustainable within the power envelope of the deployed system.
- **Software ecosystem.** Runtime environments are typically limited to lightweight inference engines such as TensorFlow Lite [Google \(2023\)](#) or ONNX Runtime [Microsoft \(2023\)](#), rather than full training frameworks.

These constraints mean that a model achieving the highest validation accuracy in a desktop experiment is not automatically the best candidate for deployment. Model size, quantisability, and inference throughput are co-equal selection criteria alongside predictive performance. The methodology in this report is designed to produce comparable metrics across all three dimensions.

1.5 Scope and Boundaries

To maintain focus and methodological rigour, this report adheres to the following explicit scope boundaries:

1. **Single-port only.** The analysis is restricted to S_{11} measurements from a one-port network. Multi-port extensions involving S_{21} , S_{12} , or S_{22} are out of scope.
2. **No time-series or drift modelling.** Each trace is treated as an independent, identically distributed sample. Temporal ordering of acquisitions, sensor drift, and sequential decision-making are not considered.
3. **No experimental results.** This document defines the methodological framework—representations, preprocessing pipelines, candidate algorithms, and evaluation protocols. Trained model performance numbers are deferred entirely to the companion experiment report.
4. **No manuscript-ready claims.** Because no experiments have been executed, no claims of superiority, state-of-the-art performance, or benchmark comparisons are made.
5. **Closed-set assumption.** The classifier is designed for the three known classes only. Detection of novel or anomalous surface states (out-of-distribution detection) is noted as a future direction but is not addressed here.

2 Dataset Abstraction

This section documents the structure, content, and integrity of the available measurement data. Precise characterisation of the dataset is a prerequisite for every downstream decision—from representation design (Section 3) to evaluation protocol—because silent assumptions about file format, frequency alignment, or class balance can propagate as systematic errors through the entire pipeline.

2.1 File Format: Touchstone .s1p

The Touchstone format is the de facto standard for exchanging frequency-domain network-parameter data between microwave instruments and simulation tools [IBIS Open Forum \(2009\)](#); [Keysight Technologies \(2017\)](#). A compliant Touchstone file begins with comment lines (prefixed by `!`) followed by an *option line* of the form

```
# <freq_unit> <param> <format> <R> <ref_impedance>
```

which declares the frequency unit, parameter type (S , Y , Z , etc.), data format (DB, MA, or RI), and reference impedance. For a single-port .s1p file the data section then contains one row per frequency point with three columns: frequency, and two format-dependent values (e.g. magnitude in dB and phase in degrees for the DB format).

2.1.1 Observed Deviation from the Standard

The 1603 files in this dataset *lack* the standard `# . . .` option header line. Each file begins directly with numeric data in a three-column, tab-separated layout with no comment lines and no header row. While the data content is consistent with the Touchstone RI (Real-Imaginary) format, the absence of the option line means that:

1. The frequency unit, data format, and reference impedance must be *inferred* rather than parsed from metadata.
2. Standard Touchstone parsers (e.g. `scikit-rf`) may reject or misinterpret these files without a preprocessing step that injects the missing header.

A robust ingestion pipeline should therefore prepend a synthetic option line—e.g. `# HZ S RI R 50`—before passing files to any standards-compliant reader.

2.2 Column Interpretation

Each data row contains three tab-separated fields:

Column	Quantity	Unit
1	Frequency	Hz
2	$\text{Re}[S_{11}]$ (Real part)	linear / unitless
3	$\text{Im}[S_{11}]$ (Imag part)	linear / unitless

All three quantities are stored as decimal floating-point numbers. Frequency values include sub-hertz fractional digits (e.g. 8000999750.000), reflecting the instrument’s internal frequency synthesis resolution.

Table 1: First five and last three rows of `Trc0.s1p` (`dry` class) assuming RI format. Frequency is in Hz, followed by Real and Imaginary linear components.

Row	Frequency (Hz)	Real ($\text{Re}[S_{11}]$)	Imaginary ($\text{Im}[S_{11}]$)
1	8 000 000 000.000	0.065	−0.013
2	8 000 999 750.000	0.066	−0.012
3	8 001 999 500.000	0.063	−0.010
4	8 002 999 250.000	0.065	−0.006
5	8 003 999 000.000	0.062	−0.006
3999	11 997 000 750.000	0.098	−0.007
4000	11 998 000 500.000	0.100	−0.003
4001	11 999 000 250.000	0.099	−0.011

2.3 Sample Trace Inspection

Table 1 reproduces the first five and last three rows of the file `Trc0.s1p` from the `dry` sensor folder. This trace is representative of the general file structure across all 1603 samples.

Several observations follow from Table 1:

- The frequency grid begins at exactly 8.0 GHz and ends at approximately 12.0 GHz (11 999 000 250 Hz), with a nominal step of $\Delta f \approx 999\,750\text{ Hz} \approx 1\text{ MHz}$.
- The step size is not an exact integer number of hertz, consistent with a VNA synthesiser dividing a 3.999 GHz span into exactly 4000 equal intervals.
- Real and Imaginary components span $[-1, 1]$ bounds, typical of a passive reflective structure.
- Because the representation is Cartesian, the curves are mathematically continuous; there are no “phase wrap” discontinuities requiring heuristic unwrapping logic.

2.4 Frequency Grid Validation

A uniform frequency grid is a critical assumption for all downstream feature representations. If any file deviates in the number of points or in the frequency values themselves, sample-wise operations (concatenation, differencing, normalisation) would silently produce misaligned features.

The expected grid is defined by three parameters:

$$\begin{aligned}
 f_{\text{start}} &= 8\,000\,000\,000\text{ Hz}, \\
 f_{\text{stop}} &= 11\,999\,000\,250\text{ Hz}, \\
 N &= 4001\text{ points},
 \end{aligned} \tag{4}$$

yielding a step of $\Delta f = (f_{\text{stop}} - f_{\text{start}})/(N - 1) \approx 999\,750.0625\text{ Hz}$. The ingestion pipeline must verify, for every file, that (i) the number of rows equals 4001, and (ii) the frequency column matches the reference grid to within a tolerance of $\pm 1\text{ Hz}$. Files failing either check should be flagged and excluded rather than silently truncated or interpolated.

2.5 Class Distribution

Table 2 summarises the number of traces per class and quantifies the imbalance.

The imbalance ratio of 1.20:1:1 is mild by the standards of applied machine learning. The `dry` class has 100 additional traces relative to `wet` and `ice`, which are equal in size. At this level of imbalance, standard stratified splitting and stratified cross-validation are typically sufficient to prevent biased evaluation. Aggressive oversampling or synthetic minority generation (e.g. SMOTE) is unlikely to be necessary but remains available as a contingency.

Table 2: Class distribution across the three surface-state categories. The imbalance ratio is computed relative to the minority classes.

Class	Folder name	Traces	Ratio
dry	dry sensor	601	1.20
wet	wet sensor	501	1.00
ice	ice on sensor	501	1.00
Total		1603	

2.6 Trace Numbering and Missing Acquisitions

File names follow the pattern `Trc<N>.s1p` where N is an integer index. Inspection of the full file listing reveals two systematic gaps in the numbering sequence:

- **Trc601–Trc699** (99 indices) are absent across all class folders.
- **Trc1201–Trc1449** (249 indices) are absent across all class folders.

These gaps are consistent across the three class directories, suggesting a measurement-campaign-level event (e.g. instrument recalibration, paused acquisition, or deliberate exclusion of a batch) rather than per-class data loss. The gaps should be noted in any publication describing the dataset but do *not* affect the machine learning pipeline, which operates on the files that are present and assigns canonical indices independently of the original trace numbers.

2.7 Folder-to-Label Mapping

Class labels are derived directly from the folder hierarchy. Each measurement file resides in one of three directories whose names serve as ground-truth labels:

Folder name	Assigned label
dry sensor	dry
wet sensor	wet
ice on sensor	ice

There is no auxiliary label file; the folder membership *is* the label. This simple scheme avoids label-file synchronisation errors but means that any accidental file movement would silently corrupt the ground truth. The ingestion pipeline should therefore checksum the full file listing against a frozen manifest at the start of every experiment.

2.8 Deterministic Indexing Scheme

To ensure reproducibility, we propose a canonical sample ID that is independent of the original trace numbering. Each sample receives an identifier of the form

$$\langle \text{class_prefix} \rangle_ \langle \text{zero-padded index} \rangle$$

where:

- **class_prefix** is one of `dry`, `wet`, or `ice`.
- **zero-padded index** is a four-digit integer assigned by sorting the files within each class folder in *lexicographic order* by their original file name, then numbering from 0000.

For example, the first file in the `dry sensor` folder (`Trc0.s1p`) receives the canonical ID `dry_0000`; the last file in the `ice on sensor` folder receives `ice_0500`. This scheme satisfies three requirements:

1. **Uniqueness.** Every sample has a globally unique identifier.
2. **Determinism.** The same set of files always produces the same mapping, regardless of operating-system directory traversal order, provided lexicographic sorting is used.
3. **Traceability.** The original file name is recorded alongside the canonical ID in the dataset manifest, preserving the link to the raw acquisition.

2.9 Metadata Logging Requirements

Every experimental run must record the following metadata to enable post-hoc auditing and reproduction:

1. **Dataset manifest:** A table mapping each canonical sample ID to its original file path and an integrity hash (e.g. SHA-256 of the raw file bytes).
2. **Frequency grid reference:** The exact f_{start} , f_{stop} , and N used, plus the tolerance applied during grid validation.
3. **Class distribution:** The number of samples per class after any exclusion or filtering step.
4. **Exclusion log:** A list of any files excluded during ingestion, with the reason for exclusion (e.g. wrong number of rows, frequency grid mismatch, corrupt data).
5. **Software versions:** Python interpreter version and versions of all critical libraries (NumPy, scikit-learn, TensorFlow/PyTorch, XGBoost, etc.).
6. **Random seeds:** All seeds used for splitting, initialisation, and any stochastic algorithm component.
7. **Preprocessing parameters:** Normalisation statistics (mean, standard deviation, min, max) computed on the training fold, along with any augmentation or feature-engineering settings.

These requirements are not optional conveniences; in a small-sample regime where every split decision materially affects reported performance, full provenance is the only defence against irreproducible results.

3 Representation Routes

The raw measurement—a pair of real-valued vectors (magnitude and phase) over 4001 frequency points—must be transformed into a feature representation before it can be consumed by a learning algorithm. The choice of representation is not neutral: it determines which spectral information is preserved, which is discarded, and which artefacts are introduced. This section develops two complementary representation families, designated *Route A* (1D spectral) and *Route B* (2D image-like), and evaluates their trade-offs systematically.

3.1 Route A: 1D Spectral Representations

Route A treats each S_{11} frequency sweep as a one-dimensional feature vector whose elements correspond to uniformly spaced frequency bins. The spectral ordering is preserved, and no spatial rasterisation is performed. Unlike traditional polar representations (Magnitude/Phase) which suffer from artificial discontinuities due to phase wrapping, this methodology strictly employs the **Cartesian representation** (Real and Imaginary components).

3.1.1 A1: Dual-Channel Cartesian Vector

The canonical 1D encoding preserves the complex nature of S_{11} by treating the Real and Imaginary components as two parallel channels:

$$\mathbf{x}_{\text{dual}} = \begin{bmatrix} \text{Re}[S_{11}(f_1)], \dots, \text{Re}[S_{11}(f_{4001})] \\ \text{Im}[S_{11}(f_1)], \dots, \text{Im}[S_{11}(f_{4001})] \end{bmatrix} \in \mathbb{R}^{2 \times 4001}. \quad (5)$$

This 2×4001 tensor is the natural input format for one-dimensional convolutional architectures (such as 1D CNN and ResNet-1D). The network receives the data analogously to a stereo audio signal. Because convolution is a linear operator, sliding a filter across the dual-channel input allows the network to naturally learn linear combinations of the Real and Imaginary components—effectively performing operations in the complex plane natively.

3.1.2 A2: Flattened Cartesian Vector

For classical machine learning algorithms (e.g., Logistic Regression, Random Forest, Gradient Boosted Trees) that expect a one-dimensional feature vector, the dual-channel matrix is flattened:

$$\mathbf{x}_{\text{flat}} = [\text{Re}[S_{11}(f_1)], \dots, \text{Re}[S_{11}(f_{4001})], \text{Im}[S_{11}(f_1)], \dots, \text{Im}[S_{11}(f_{4001})]]^T \in \mathbb{R}^{8002}. \quad (6)$$

This encoding provides exactly the same information as A1, but the spatial co-location of the Real and Imaginary components at the same frequency is lost to the algorithm (except for tree-based models which can route based on independent features). The high dimensionality ($d = 8002$) relative to the sample size ($N = 1603$) necessitates strong regularisation to prevent overfitting in linear models.

3.1.3 A3: Derivative Augmented Cartesian Vector

Spectral derivatives highlight *local* shape features—slopes, inflection points, and resonance edges—that may be more discriminative than the absolute amplitude at any single frequency. The first-order finite difference approximates the spectral derivative for both components:

$$\Delta_k^{\text{Re}} = \frac{\text{Re}[S_{11}(f_{k+1})] - \text{Re}[S_{11}(f_k)]}{\Delta f}, \quad (7)$$

$$\Delta_k^{\text{Im}} = \frac{\text{Im}[S_{11}(f_{k+1})] - \text{Im}[S_{11}(f_k)]}{\Delta f}. \quad (8)$$

These differences are appended to the flat vector or concatenated as additional channels in the tensor representation. While first differences are invariant to additive baseline shifts (providing robustness against calibration drift), they amplify high-frequency instrumental noise.

3.1.4 Route A Encoding Summary

Table 3 compares the three 1D Cartesian encoding variants.

Table 3: Comparison of Route A (1D spectral) Cartesian encoding variants.

ID	Encoding	Dim.	Model Compatibility
A1	Dual-Channel Cartesian	2×4001	1D CNN, ResNet-1D
A2	Flattened Cartesian	8002	Logistic Regression, SVM, GBT, RF, MLP
A3	Derivative Augmented	16004	All (with higher overfitting risk)

3.2 Route B: 2D Image-Like Representations

Route B converts each spectral trace into a two-dimensional image, enabling the use of established convolutional neural network architectures originally designed for visual recognition tasks. The conversion introduces a *rendering step* that is absent in Route A, and the properties of that rendering step materially affect downstream classification.

3.2.1 B1: Grayscale Curve Images

The most direct conversion plots one of the Cartesian traces (e.g. $\text{Re}[S_{11}(f)]$) as a curve against frequency on a white background, then rasterises the plot to a fixed pixel resolution. Typical target resolutions are 128×128 or 224×224 pixels, the latter matching the default input size of many pretrained image classifiers (e.g. ResNet, VGG, MobileNet).

The resulting single-channel (grayscale) image encodes:

- The *vertical position* of the curve at each horizontal pixel $\Leftrightarrow$ the amplitude of the chosen component at the corresponding frequency.
- The *shape* of the curve $\Leftrightarrow$ local spectral features (dips, slopes, inflections).

However, the rendering process introduces several artefacts that have no physical origin in the measurement:

- **Line thickness and anti-aliasing.** The width of the plotted curve in pixels, and the sub-pixel smoothing applied by the rendering engine, create intensity gradients that a CNN may learn to exploit. These gradients are properties of the rendering configuration, not of the sensor response.
- **Axis scaling.** The choice of amplitude axis range (e.g. $[-1.0, 1.0]$ vs. $[-0.2, 0.2]$) affects how much of the image canvas the curve occupies. A tight axis range amplifies small variations; a wide range compresses them into fewer pixels.
- **Background and margins.** Axis labels, tick marks, grid lines, and border padding consume pixels that carry no measurement information but may correlate with class if rendering parameters are inadvertently class-dependent.

To mitigate these risks, the rendering pipeline must use *identical* axis ranges, line styles, and canvas dimensions for every sample, and should strip all non-data elements (axes, labels, borders) from the final image.

3.2.2 B2: Dual-Channel Rasterised Maps

Analogous to encoding A1, one may render Real and Imaginary components as two separate image channels, producing a $H \times W \times 2$ tensor. Each channel is a grayscale curve image as described in Section 3.2.1, plotted with independently normalised axis ranges.

This retains the full complex information in image form, and because the Cartesian components are continuous, it neatly avoids the sharp vertical jumps that would otherwise be caused by wrapped-phase discontinuities in polar representations.

3.2.3 B3: Spectrogram-Like and Heatmap Constructions

A more creative approach constructs a 2D image not by plotting a curve but by *tiling* or *reshaping* the spectral data. Several strategies are conceivable:

1. **Frequency-axis interpolation.** The 4001-point Cartesian vector is reshaped into a 2D matrix (e.g. 63×63 plus padding, or 50×80) and rendered as a heatmap. The row and column indices are arbitrary partitions of the frequency axis, and the pixel intensity encodes amplitude.
2. **Multi-representation tiling.** Real, Imaginary, and derivative traces are rendered as separate horizontal strips and stacked vertically into a single image, creating a composite “spectrogram-like” representation.
3. **Recurrence-plot or Gramian-field encoding.** The spectral vector is transformed into a square matrix via a pairwise operation (e.g. Gramian Angular Field), producing a texture image that encodes temporal/spectral correlations.

These approaches can expose inter-frequency relationships that a 1D convolution might miss, but they introduce a significant design burden: the reshaping geometry, colour mapping, and interpolation method all become hyperparameters that are *ad hoc* with respect to the measurement physics. Unless a clear empirical advantage is demonstrated, the additional complexity is difficult to justify for a three-class problem with 1603 samples.

3.2.4 Rendering Artefact Warning

Warning. Every Route B representation injects information into the model’s input that is not present in the original measurement physics. Line thickness, anti-aliasing kernels, colour-map transfer functions, axis scaling, and pixel resolution are all rendering decisions, not sensor observations. A CNN trained on Route B images may learn features that correlate with class labels only because of consistent rendering choices rather than genuine spectral differences. This risk is mitigated by strict rendering standardisation but can never be fully eliminated. Route B results should always be compared against Route A baselines to assess whether the image pathway adds genuine discriminative value or merely introduces a confound.

3.3 Route A vs. Route B: Comparative Summary

Table 4 provides a structured comparison of the two representation families across four evaluation axes.

3.4 Representation Selection Strategy

Given the characteristics summarised in Table 3 and Table 4, the experimental campaign should follow a principled selection strategy:

1. **Start with Route A.** Establish baseline performance using the Flattened Cartesian encoding (A2) with classical models, and Dual-Channel Cartesian encoding (A1) with 1D CNNs. This provides a low-complexity reference against which all subsequent representations and algorithms are compared.
2. **Incrementally add information.** Evaluate A3 (Derivative Augmented) to quantify the marginal value of local-shape features.

Table 4: Comparison of Route A (1D spectral) and Route B (2D image-like) representation families.

Axis	Route A (1D Spectral)	Route B (2D Image-Like)
Information fidelity	Lossless. Every frequency-bin value is preserved at full floating-point precision.	Lossy. Rasterisation maps continuous-valued amplitudes to discrete pixel intensities, and spatial quantisation limits the recoverable frequency resolution.
Computational cost	Low. No rendering step required. Preprocessing is limited to normalisation and optional feature engineering.	Moderate to high. Rendering each trace to an image adds I/O and CPU time. Larger image resolutions (224×224) increase both storage and training cost.
Model compatibility	Compatible with all classical models (flat vector) and 1D neural architectures (1D CNN, ResNet-1D, MLP, KAN). Incompatible with 2D CNNs without reshaping.	Compatible with 2D CNNs and pretrained image models. Incompatible with classical models unless flattened (at extreme dimensionality, e.g. $224 \times 224 = 50\,176$).
Artefact risk	Negligible. No rendering step means no rendering artefacts. Preprocessing artefacts are explicit and auditable.	Significant. Line thickness, anti-aliasing, axis scaling, colour mapping, and background elements are all potential confounds. Requires strict standardisation and ablation.

3. **Introduce Route B only if justified.** Route B should be explored if Route A performance plateaus and there is a hypothesis that 2D spatial patterns (e.g. inter-frequency correlations visible in heatmaps) contain discriminative information that 1D convolutions cannot capture.
4. **Always compare against Route A.** Any Route B result must be benchmarked against the corresponding Route A encoding to disentangle genuine spectral information from rendering artefacts.

This incremental strategy avoids premature commitment to high-complexity representations and ensures that every added dimension of information earns its place through measurable performance improvement.

4 Preprocessing Pipeline

Before any learning algorithm receives data, every S_{11} trace must pass through a deterministic preprocessing pipeline that ensures correctness, consistency, and traceability. This section describes *Stage 1* of a three-stage data-conditioning workflow; Section 5 and Section 7 cover the subsequent stages.

4.1 Three-Stage Pipeline Overview

The full data-conditioning workflow is decomposed into three strictly sequential stages, each with a distinct purpose:

- Stage 1: Validity preprocessing** (this section). Parses raw files, verifies structural and numerical integrity, quarantines malformed or outlier traces, and assigns canonical sample identifiers. At the end of this stage every surviving sample is a well-formed numerical array with a unique ID and a verified class label.
- Stage 2: Normalization** (Section 5). Transforms feature values (magnitude, phase, or pixel intensities) to distributions that accelerate and stabilise learning. Normalization statistics are computed on the training partition *only* and applied identically to validation and test partitions.
- Stage 3: Augmentation** (Section 7). Synthetically expands the training set with physics-aware perturbations at training time only. Augmentation is never applied to validation or test data.

Figure 1 sketches the three stages and their data-flow boundaries.

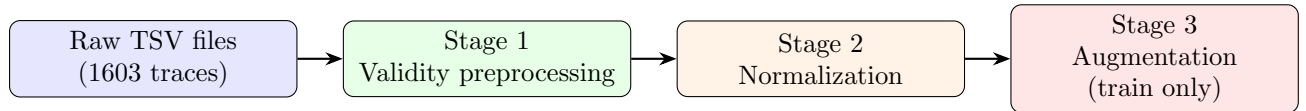


Figure 1: Three-stage data-conditioning pipeline. Stages 2 and 3 are covered in Sections 5 and 7, respectively.

The remainder of this section details every sub-step of Stage 1.

4.2 Touchstone File Parsing

Each S_{11} trace is stored as a plain-text file with three tab-separated columns and no header row:

Column 1	Column 2	Column 3
Frequency (Hz)	$ S_{11} $ (dB)	$\angle S_{11}$ (deg)

Parsing assumptions.

- **Delimiter:** horizontal tab (`\t`). The parser must not silently accept comma- or space-delimited variants; mismatched delimiters indicate a file-format error.
- **No header row:** the first line is expected to be numeric data. However, a small number of files may carry an optional Touchstone-style option line beginning with `# IBIS Open Forum` (2009). The parser shall skip any line whose first non-whitespace character is `#` and log the event.
- **Column count:** exactly three numeric fields per non-comment line. Lines with fewer or more fields trigger a malformed-row error.
- **Encoding:** UTF-8 or ASCII; reject files with binary or non-decodable content.

Column interpretation. After parsing, each trace is a matrix $\mathbf{T} \in \mathbb{R}^{4001 \times 3}$ with columns $(f, |S_{11}|_{\text{dB}}, \angle S_{11})$. The frequency column is in hertz; for readability in plots and tables we convert to gigahertz (GHz) by dividing by 10^9 , yielding a range of approximately 8.0–12.0 GHz with a step of roughly 1 MHz. The magnitude column is already in decibels (a logarithmic scale) and requires no further transformation at this stage. The phase column is in degrees and is retained as-is; conversion to radians is deferred to the normalization stage if required by a specific algorithm.

4.3 Frequency-Grid Validation

A fundamental assumption of the downstream pipeline is that every trace shares the *same* $N = 4001$ -point frequency vector. This enables treating each frequency bin as a fixed feature across all samples. The validation step proceeds as follows:

1. Parse the frequency column of the first valid file and designate it the *reference grid* $\mathbf{f}_{\text{ref}}$.
2. For every subsequent file, compute the element-wise absolute difference $\Delta_i = |f_i - f_{\text{ref},i}|$ for $i = 1, \dots, 4001$.
3. If $\max_i \Delta_i > \epsilon$ (with $\epsilon = 1$ Hz), the file is flagged as *grid-mismatched* and quarantined with a diagnostic log entry.
4. Files with $N \neq 4001$ points are similarly quarantined.

Grid mismatches could indicate a different VNA frequency plan, a truncated sweep, or a data-export error. Quarantined files are excluded from all subsequent processing but retained on disk for manual inspection.

4.4 Missing-Value and Malformed-File Handling

Missing and non-finite values. Each parsed numeric entry is checked for NaN, $\pm\infty$, and non-numeric content (e.g., text strings in a numeric column). The handling policy is:

- **Isolated NaN/Inf** (fewer than 0.1 % of points in a trace): replace via linear interpolation from the two nearest valid neighbours in the frequency dimension. Log the location and original value.
- **Widespread NaN/Inf** (≥ 0.1 % of points): reject the entire file. Interpolating large gaps risks introducing artefacts that distort class-discriminative spectral features.

Malformed files. The following conditions result in immediate file rejection:

- Fewer than 4001 non-comment lines (truncated acquisition).
- Any non-comment line with a column count $\neq 3$.
- File size of zero bytes or failure to decode as text.

Every rejection is logged with the file path, the reason, and a timestamp. The total number of rejected files is reported before proceeding.

4.5 Outlier Inspection

Outlier screening is performed *before* training to detect anomalous traces that may represent measurement errors, sensor malfunctions, or mislabelled files. The procedure operates per-frequency-bin within each class:

1. For each class $c \in \{\text{dry}, \text{wet}, \text{ice}\}$ and each frequency index $i = 1, \dots, 4001$, compute the mean $\mu_{c,i}$ and standard deviation $\sigma_{c,i}$ of the magnitude values across all samples in class c .
2. Flag any sample whose magnitude at bin i satisfies

$$|x_i - \mu_{c,i}| > 4\sigma_{c,i}$$

as a candidate outlier at that bin.

3. A sample flagged at more than 1% of frequency bins is promoted to a *trace-level outlier candidate*.
4. Trace-level candidates are *not* automatically removed; they are presented for visual inspection via class-aggregated envelope plots (mean $\pm 2\sigma$ and $\pm 4\sigma$ bands), and the analyst decides on retention, relabelling, or exclusion.

The same procedure is repeated for the phase channel. Automatic removal without human review is deliberately avoided because a trace that appears statistically anomalous may represent a genuine edge case that the classifier should learn.

4.6 Class-Label Mapping

Class labels are derived from the folder structure of the dataset:

Folder name	Integer label	Sample count
dry	0	601
wet	1	501
ice	2	501
Total		1603

The class imbalance ratio is $601 : 501 : 501 \approx 1.20 : 1 : 1$, which is mild but non-trivial; it is accounted for during evaluation (??) via macro-averaged metrics.

Trace-numbering gaps. Inspection of the raw file names reveals numbering gaps in the trace indices: traces Trc601–Trc699 and Trc1201–Trc1449 are absent across all classes. These gaps are noted as potential missing acquisitions or intentional exclusions by the data provider. The pipeline does not attempt to impute or reconstruct the missing trace indices; it simply uses the files that are present.

4.7 Deterministic Sample Indexing

To ensure full reproducibility across experiments, each surviving sample is assigned a canonical integer identifier:

1. Sort all valid file paths lexicographically within each class folder.
2. Concatenate the sorted lists in label order ($0 \rightarrow 1 \rightarrow 2$).
3. Assign sequential IDs starting from 0.

A mapping file `sample_index.csv` is written with columns `sample_id`, `class_label`, `class_name`, and `original_filepath`. This file is version-controlled alongside the code and referenced in all subsequent logs to ensure that any experimental result can be traced back to the exact input files.

4.8 Stage 1 Summary

Table 5 summarises the validity checks applied during Stage 1.

Table 5: Stage 1 validity-preprocessing checklist.

Check	Condition	Action on failure
Comment-line filtering	Line starts with #	Skip line, log event
Column count	$\neq 3$ fields per line	Reject file
Row count	$\neq 4001$ data rows	Quarantine file
Frequency grid	$\max \Delta f > 1$ Hz vs. reference	Quarantine file
Non-finite values	NaN or $\pm\infty$ entries	Interpolate ($< 0.1\%$) or reject
Outlier screening	$> 4\sigma$ in $> 1\%$ of bins	Flag for visual review

After Stage 1, the dataset consists of $N_{\text{valid}} \leq 1603$ well-formed, grid-aligned, canonically indexed traces ready for normalization (Section 5).

5 Normalization

Normalization is **Stage 2** of the data-conditioning pipeline introduced in Section 4. Its purpose is to rescale feature values so that learning algorithms converge faster and are less sensitive to the raw physical units of the input channels. Every normalization method discussed here obeys a single inviolable rule: *statistics are computed on the training partition only and applied identically to validation and test partitions.*

5.1 Anti-Leakage Principle

Warning — Data-Leakage Risk

Fitting normalization statistics (mean, standard deviation, minimum, maximum, median, or interquartile range) on the *entire* dataset — including validation and test partitions — constitutes **data leakage**. The resulting performance estimates are optimistically biased because the model indirectly observes distributional information from held-out samples during training. All normalization parameters must be computed exclusively on the training fold and then frozen before transforming any held-out data.

In a cross-validation setting (Section 6), this means normalization statistics are *re-computed* for every fold: the inner training set defines the transform, and the inner validation set (and outer test set, in nested CV) receives the same frozen transform.

5.2 Per-Feature Z-Score Normalization

Z-score (standard-score) normalization centres each feature at zero mean and unit variance:

$$x'_j = \frac{x_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}}, \quad (9)$$

where j indexes the feature (i.e., the frequency bin or pixel position), and $\mu_j^{\text{train}}, \sigma_j^{\text{train}}$ are the mean and standard deviation computed over the training partition.

When to prefer z-score. Z-score normalization is appropriate when the feature distributions are approximately Gaussian or at least unimodal and symmetric. It is the default choice for algorithms that assume or benefit from standardised inputs, including Logistic Regression [Hosmer et al. \(2013\)](#), Linear SVM [Vapnik \(1995\)](#); [Cortes and Vapnik \(1995\)](#), and Multilayer Perceptrons [Haykin \(2009\)](#); [Goodfellow et al. \(2016\)](#).

Sensitivity to outliers. Because μ and σ are influenced by extreme values, z-score normalization can compress the bulk of the distribution when heavy-tailed outliers are present. If the outlier inspection in Section 4 reveals residual anomalies, robust scaling (Section 5.4) is preferred.

5.3 Min-Max Scaling

Min-max scaling maps each feature linearly into the interval $[0, 1]$:

$$x'_j = \frac{x_j - x_{j,\min}^{\text{train}}}{x_{j,\max}^{\text{train}} - x_{j,\min}^{\text{train}}}. \quad (10)$$

When to prefer min-max. Min-max scaling is useful when a bounded output range is desirable, for example as input to layers with sigmoid or softmax activations, or when converting traces to greyscale pixel values in $[0, 255]$ for Route B image representations (Section 3). It preserves zero entries and the shape of the original distribution, but it is *highly* sensitive to outliers: a single extreme value can compress all other values into a narrow sub-interval.

Out-of-range values at test time. Because the min and max are determined from training data, test-time values may fall outside $[0, 1]$. The recommended policy is to *clip* to $[0, 1]$ and log any clipped samples rather than silently extrapolating.

5.4 Robust Scaling

When outlier contamination is suspected — or confirmed by the 4σ screening in Section 4 — robust scaling replaces mean and standard deviation with the median ($\tilde{x}$) and interquartile range (IQR):

$$x'_j = \frac{x_j - \tilde{x}_j^{\text{train}}}{\text{IQR}_j^{\text{train}}}, \quad \text{IQR}_j = Q_{75,j} - Q_{25,j}. \quad (11)$$

Robust scaling does not bound the output range, but it ensures that the central 50% of the data occupies a unit-width interval regardless of tail behaviour. It is compatible with all algorithms considered in this report and is recommended as a default for exploratory runs before the outlier landscape is fully characterised.

5.5 Channel-Wise Normalization for Route A

Route A (1D spectral) represents each sample as a dual-channel vector: one channel for $|S_{11}|$ (dB) and one for $\angle S_{11}$ (deg). The two channels have fundamentally different scales — magnitude values typically range from 0 dB to -40 dB, while phase values span $\pm 180^\circ$ — and must therefore be normalised *independently*.

Concretely, if a sample is represented as $\mathbf{x} = [\mathbf{m}; \mathbf{p}] \in \mathbb{R}^{2 \times 4001}$, the normalization statistics (μ, σ) or $(x_{\min}, x_{\max})$ are computed separately for the magnitude slice $\mathbf{m}$ and the phase slice $\mathbf{p}$ across the training partition. This is the one-dimensional analogue of channel-wise image normalization and is standard practice in multi-variate time-series classification [Goodfellow et al. \(2016\)](#).

5.6 Image Normalization for Route B

Route B converts each S_{11} trace into a 2D image-like representation (e.g., a magnitude–frequency heatmap or a recurrence-plot image). Normalization follows the conventions established in the computer-vision literature:

- **Single-channel images:** compute pixel-level mean μ_{px} and standard deviation σ_{px} across all training images, then apply Equation (9) pixel-wise.
- **Dual-channel images** (magnitude and phase as separate channels): compute (μ, σ) per channel across the training set, paralleling standard RGB-channel normalization.

Batch normalization [Ioffe and Szegedy \(2015\)](#) provides an additional, learnable normalization layer inside deep networks. It normalises activations within each mini-batch during training and uses running estimates at inference time. While batch normalization is architecture-internal rather than a preprocessing step, it interacts with the input normalization: applying both z-score input normalization and batch normalization is common and generally beneficial for training stability.

5.7 Comparison of Normalization Strategies

Table 6 summarises the key properties of each normalization method.

Table 6: Comparison of normalization strategies.

Method	Assumptions	Outlier sensitivity	Output range	Route compat.
Z-score	Approx. Gaussian features	High	Unbounded	A, B
Min-max	Bounded, no extreme outliers	Very high	$[0, 1]$	A, B
Robust (median/IQR)	Arbitrary; tolerates tails	Low	Unbounded	A, B
Batch norm [†]	Internal to deep nets	Moderate	Learned	A (1D CNN, ResNet-1D), B (2D CNN)

[†] Batch normalization is an architecture component, not a preprocessing step [Ioffe and Szegedy \(2015\)](#).

5.8 Practical Recommendations

1. **Default:** begin with per-feature z-score normalization (Equation (9)) for all Route A experiments and pixel-level z-score for Route B. It is well-understood, widely supported by scikit-learn [Pedregosa et al. \(2011\)](#) and TensorFlow, and compatible with every algorithm in ??.
2. **If outliers persist** after the screening in Section 4, switch to robust scaling (Equation (11)). Compare validation performance against z-score to confirm the benefit.
3. **If bounded inputs are required** (e.g., for sigmoid output layers or greyscale image encoding), use min-max scaling (Equation (10)) with post-hoc clipping at test time.
4. **Always verify:** after normalization, inspect the per-feature distributions (histograms or Q-Q plots) on a held-out fold to confirm that no pathological compression or inflation has occurred.
5. **Record the transform:** serialise the fitted scaler object (e.g., via `joblib` or ONNX [Microsoft \(2023\)](#)) so that identical normalization can be applied at inference time on the edge device.

6 Overfitting Control and Evaluation Integrity

With only 1603 labelled traces, this project operates in a regime where overfitting and evaluation bias are the *primary* threats to credible conclusions. A model that memorises the training set — or a cross-validation procedure that leaks information across folds — can produce accuracy figures that bear no relationship to real-world performance on the edge sensor. This section catalogues the safeguards that the experimental pipeline must enforce.

6.1 Context: Small-Sample Deep Learning

Modern deep architectures routinely contain millions of trainable parameters. A ResNet-1D or 1D CNN with even a modest channel depth can easily exceed 10^6 weights, yielding a parameter-to-sample ratio far above unity. Classical statistical learning theory warns that generalisation degrades when model capacity greatly exceeds the information content of the training set [Vapnik \(1995\)](#). While modern regularisation techniques mitigate this to a degree, they do not eliminate the risk.

The practical consequence is twofold:

1. **Architecture selection** should start from the smallest viable model and scale up only if validation performance saturates (Section 6.6).
2. **Evaluation protocols** must be unusually rigorous; a single random train/test split is insufficient for stable performance estimates when $N = 1603$ (Section 6.3).

6.2 Stratified Train / Validation / Test Split

The simplest holdout protocol divides the dataset into three disjoint partitions while preserving the class proportions ($601 : 501 : 501 \approx 1.20:1:1$):

Partition	Share	Purpose
Training	60–70 %	Fit model parameters
Validation	15–20 %	Tune hyper-parameters, early stopping
Test	15–20 %	Final, single-use performance estimate

Stratification ensures that every partition contains approximately the same proportion of **dry**, **wet**, and **ice** samples. For $N = 1603$ and a 60/20/20 split, the test set contains roughly 320 samples — enough for per-class precision and recall estimates but small enough that a single unlucky split can shift accuracy by several percentage points.

6.3 Repeated Stratified k -Fold Cross-Validation

To obtain more stable performance estimates, we employ *repeated stratified k -fold cross-validation* [Kohavi \(1995\)](#). Two configurations are considered:

- **5×5 -fold:** five repetitions of 5-fold CV, yielding 25 performance measurements. Each fold contains ~ 320 test samples.
- **3×10 -fold:** three repetitions of 10-fold CV, yielding 30 measurements with ~ 160 test samples per fold.

The mean and standard deviation (or 95 % confidence interval) across all folds are reported. A high standard deviation signals that the model’s performance is sensitive to the particular train/test partition, which is itself a diagnostic of overfitting or insufficient data.

Critical implementation detail. All preprocessing that involves fitting to data — normalization statistics (Section 5), feature-extraction parameters, and dimensionality-reduction transforms — must be fit *exclusively* on the training fold of each CV iteration and re-fit from scratch when the fold changes. Fitting once on the full dataset and then splitting constitutes data leakage and invalidates the CV estimate.

6.4 Nested Cross-Validation for Hyper-Parameter Tuning

When hyper-parameters (e.g., regularisation strength, learning rate, tree depth) are tuned via grid or random search, an additional inner CV loop is required to prevent the test-fold performance from being confounded with the tuning process. This is *nested cross-validation* [Varma and Simon \(2006\)](#); [Cawley and Talbot \(2010\)](#):

1. **Outer loop** (k_{out} folds): reserves one fold as the test set.
2. **Inner loop** (k_{in} folds on the remaining data): selects the best hyper-parameter configuration by validation performance.
3. The best configuration from the inner loop is retrained on the full outer-training set and evaluated on the outer test fold.

A typical configuration is $k_{\text{out}} = 5$, $k_{\text{in}} = 3$, yielding 15 model fits per hyper-parameter candidate. The computational cost is substantial but necessary: Cawley and Talbot [Cawley and Talbot \(2010\)](#) demonstrated that non-nested CV can overfit the *evaluation metric itself*, producing optimistic estimates that do not transfer to unseen data.

6.5 Near-Duplicate and Temporal-Correlation Awareness

The trace file names (e.g., `Trc100.txt`, `Trc101.txt`) suggest sequential acquisition. If consecutive traces were recorded within seconds of each other under identical sensor conditions, they may be near-duplicates: statistically indistinguishable samples that, if split across training and test partitions, artificially inflate performance.

Detection protocol.

1. Compute the pairwise Pearson correlation (or cosine similarity) of the magnitude vectors within each class.
2. Identify clusters of traces with inter-trace correlation $r > 0.999$ (or another empirically determined threshold).
3. Assign all members of a high-correlation cluster to the *same* CV fold using a *group-aware* splitter (e.g., `GroupKFold` in scikit-learn [Pedregosa et al. \(2011\)](#)).

Group-aware splitting ensures that the model cannot trivially recognise a test trace by having seen its near-twin during training. The trace- numbering gaps noted in Section 4 (`Trc601–Trc699`, `Trc1201–Trc1449`) may further indicate batch boundaries that should be respected during splitting.

6.6 Capacity–Dataset Size Mismatch

Table 7 provides rough parameter counts for the candidate architectures. The table is indicative; exact counts depend on final architecture choices.

Table 7: Indicative parameter counts and overfitting risk relative to $N = 1603$ samples.

Algorithm	Approx. parameters	Param./sample	Overfit risk
Logistic Regression	$\sim 12\,000$	~ 7	Low
Linear SVM	$\sim 12\,000$	~ 7	Low
RBF SVM	support vectors	—	Moderate
Random Forest	ensemble, depth-bound	—	Low–Mod
Gradient Boosted Trees	ensemble, depth-bound	—	Moderate
Multilayer Perceptron	50 000–500 000	30–300	High
1D CNN	100 000–2 000 000	60–1200	High
ResNet-1D	500 000–5 000 000	300–3000	Very high
KAN	architecture-dependent	variable	High

Recommendation. Begin every deep-learning experiment with the *smallest* viable architecture (e.g., two convolutional layers with 16–32 filters for the 1D CNN). Scale up only if validation loss plateaus without overfitting. Tree-based ensembles and linear models serve as strong baselines precisely because their effective capacity is easier to control.

6.7 Regularization Toolkit

The following regularisation techniques are available and should be applied in combination as needed.

6.7.1 Early Stopping

Monitor the validation loss at the end of each training epoch. If the loss does not improve for a predefined number of consecutive epochs (the *patience*), training is halted and the model weights from the best epoch are restored. A patience of 10–20 epochs is a reasonable starting point for this dataset size; excessively large patience allows the model to overfit before the stopping criterion triggers.

6.7.2 Weight Decay / L2 Regularization

Weight decay adds a penalty proportional to the squared ℓ_2 -norm of the model weights to the loss function:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{task}} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (12)$$

For adaptive optimisers (Adam, AdamW), decoupled weight decay [Loshchilov and Hutter \(2019\)](#) is preferred because it disentangles the regularisation effect from the adaptive learning rate.

6.7.3 Dropout

Dropout [Srivastava et al. \(2014\)](#) randomly zeroes a fraction p of activations during each forward pass, forcing the network to learn redundant representations. Typical values are $p \in [0.2, 0.5]$. For very small datasets, moderate dropout ($p \approx 0.3$) combined with early stopping often outperforms aggressive dropout, which can under-fit.

6.7.4 Label Smoothing

Standard one-hot targets encourage the model to produce overconfident predictions (logit magnitudes $\rightarrow \infty$). Label smoothing replaces the hard target $y_c \in \{0, 1\}$ with a soft target:

$$y_c^{\text{smooth}} = (1 - \alpha) y_c + \frac{\alpha}{C}, \quad (13)$$

where $C = 3$ is the number of classes and $\alpha \in [0.05, 0.2]$ is the smoothing factor. This reduces the gradient magnitude for already-correct predictions and acts as an implicit regulariser.

6.7.5 Pruning

Post-training weight pruning removes parameters that contribute little to the output, simultaneously reducing model size and (potentially) improving generalisation by eliminating memorised noise. Pruning is especially relevant for edge deployment ([??](#)), where model size and inference latency are constrained.

6.8 Confidence Calibration as a Diagnostic

A well-calibrated model produces predicted probabilities that match empirical class frequencies: if the model assigns $p = 0.8$ to a class across many test samples, roughly 80 % of those predictions should be correct. Guo et al. [Guo et al. \(2017\)](#) showed that modern deep networks are frequently *miscalibrated* — typically overconfident — even when their accuracy is high.

Calibration diagnostics (reliability diagrams, Expected Calibration Error) are **useful post-training assessments** of prediction quality but are **not substitutes** for proper generalisation control. A model can be well-calibrated yet still overfit to a leaky evaluation protocol. Calibration should therefore be measured *after* all the safeguards in this section are in place, using the held-out test set or the outer fold of nested CV.

Post-hoc calibration methods (temperature scaling, Platt scaling) can correct miscalibration at minimal computational cost and are recommended before edge deployment, where the classifier’s confidence score may influence downstream decision logic (e.g., “flag for human review if $\max p_c < 0.7$ ”).

6.9 Summary of Safeguards

Key Principle

Small-sample edge-sensing work is *especially* vulnerable to optimistic performance estimates produced by careless data splits, leaked preprocessing, or unchecked model capacity. Every reported metric in this project must survive the following audit:

1. Was normalization fit only on the training fold?
2. Were near-duplicate traces kept in the same fold?
3. Was hyper-parameter tuning isolated in an inner CV loop?
4. Is the reported metric averaged over multiple folds/repeats?
5. Is the model’s parameter count justified relative to $N = 1603$?

A negative answer to any of these questions invalidates the associated result.

7 Data Augmentation

Augmentation is **Stage 3** of the data-conditioning pipeline introduced in Section 4. Its purpose is to synthetically expand the training set so that models — particularly deep networks with high parameter counts (Section 6) — see greater input variety during optimisation. Two principles govern every augmentation applied in this project:

1. **Physics-awareness.** Every transformation must have a credible physical or instrumental justification. S_{11} traces encode the electromagnetic response of a sensor in a specific physical state; arbitrary perturbations that violate Maxwell’s equations or produce non-physical spectral shapes do not improve real-world robustness.
2. **Training-time only.** Augmented samples are generated on-the-fly (or pre-generated and cached) for the training partition exclusively. Validation and test data are never augmented; doing so would corrupt the evaluation protocol.

Warning — Plausibility vs. Diversity

Augmentations that create visually diverse but physically implausible samples can inflate the apparent dataset size without improving real-world generalisation. Each augmentation must have a credible physical or instrumental justification. When in doubt, ablate: train with and without the augmentation and compare validation performance.

7.1 Additive Gaussian Noise

Physical justification. Every VNA measurement contains additive thermal and receiver noise. The noise floor depends on the instrument’s dynamic range, IF bandwidth, and averaging factor. Injecting Gaussian noise simulates repeated measurements of the same physical state under slightly different instrument conditions.

Implementation. For each training sample $\mathbf{x} \in \mathbb{R}^{4001}$ (magnitude channel), generate a perturbed copy:

$$\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_{\text{noise}}^2), \quad (14)$$

where σ_{noise} is chosen to match a realistic signal-to-noise ratio (SNR). For a modern benchtop VNA with a dynamic range of 80 dB or more, noise levels on the order of 0.01–0.1 dB per point are representative. The same procedure is applied independently to the phase channel with an appropriate phase-noise variance (typically a few hundredths of a degree).

Risk level: Low. Additive instrument noise is a well-understood corruption mechanism. The augmentation is compatible with both Route A and Route B.

7.2 Frequency-Local Smooth Perturbation

Physical justification. Minor calibration drift between measurement sessions can introduce smooth, low-frequency distortions in the magnitude and phase responses. These manifest as gentle undulations superimposed on the true spectral shape rather than as point-wise noise.

Implementation. Generate a smooth perturbation curve by sampling a small number ($K = 5\text{--}10$) of random control points along the frequency axis, drawing their amplitudes from $\mathcal{N}(0, \sigma_{\text{jitter}})$, and interpolating with a cubic spline to produce a continuous perturbation vector $\boldsymbol{\delta} \in \mathbb{R}^{4001}$. The perturbed trace is $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\delta}$.

The amplitude σ_{jitter} must be small enough that the resulting trace remains within the class-aggregated envelope identified during outlier screening (Section 4).

Risk level: Low–Medium. The perturbation is physically motivated but the amplitude must be bounded carefully; excessive jitter can blur inter-class boundaries.

7.3 Magnitude Scaling

Physical justification. Variations in cable loss, connector torque, and adapter insertion loss introduce a quasi-constant multiplicative offset (in log-magnitude, an additive offset in dB) across the frequency band. Simulating this variation makes the classifier robust to test-time setup differences.

Implementation. Apply a per-sample random gain g drawn uniformly from $[-1.5 \text{ dB}, 1.5 \text{ dB}]$:

$$\tilde{x}_i = x_i + g, \quad g \sim \mathcal{U}(-1.5, 1.5), \quad (15)$$

where x_i is the magnitude at frequency bin i in dB. The $\pm 1.5 \text{ dB}$ range is representative of typical cable and connector variability in X-band (8–12 GHz) setups.

Risk level: Low. A uniform gain offset preserves the spectral *shape*, which carries the class-discriminative information. Route A and Route B compatible.

7.4 Phase Perturbation

Physical justification. Electrical length changes due to cable flexure or temperature drift introduce a nearly linear phase offset $\Delta\phi(f) \approx \alpha + \beta f$ across the band. Small additional smooth fluctuations simulate connector phase instability.

Implementation. For each training sample, generate a random affine phase offset:

$$\tilde{\phi}_i = \phi_i + \alpha + \beta f_i, \quad (16)$$

where $\alpha \sim \mathcal{U}(-5^\circ, 5^\circ)$ is a constant offset and β is drawn such that the total additional phase slope across the 4 GHz band does not exceed $\pm 10^\circ$. Optionally, a smooth spline perturbation (as in Section 7.2) may be added to simulate non-linear phase instability, but the total perturbation must remain small enough to avoid creating non-physical discontinuities or phase wrapping artefacts.

Risk level: Medium. Phase perturbations are physically justified but require careful bounding. Excessive phase noise can create unrealistic spectral features, particularly near resonances where phase transitions are steep.

7.5 Frequency-Axis Shift

Physical justification. A rigid shift of the frequency axis simulates a change in the VNA’s start/stop frequencies or a systematic frequency-reference error. This augmentation is **only justified** if the sensor acquisition protocol includes variable centre-frequency settings or IF bandwidth changes that alter the effective frequency grid.

Implementation (conditional). Shift the entire trace by Δf frequency bins (with edge padding):

$$\tilde{x}_i = x_{i+\Delta f}, \quad \Delta f \sim \mathcal{U}(-N_{\text{shift}}, N_{\text{shift}}), \quad (17)$$

where N_{shift} is a small integer (e.g., 1–5 bins, corresponding to 1–5 MHz).

Risk level: Medium–High. If the acquisition protocol uses a fixed frequency plan (as appears to be the case in this dataset), frequency-axis shifting has no physical basis and should **not** be applied. It is listed here for completeness and should be reconsidered only if the data provider confirms variable acquisition settings.

7.6 Mixup Augmentation

Mixup [Zhang et al. \(2018\)](#) creates new training samples as convex combinations of existing ones:

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_a + (1 - \lambda) \mathbf{x}_b, \quad \tilde{y} = \lambda y_a + (1 - \lambda) y_b, \quad \lambda \sim \text{Beta}(\alpha, \alpha), \quad (18)$$

where $(\mathbf{x}_a, y_a)$ and $(\mathbf{x}_b, y_b)$ are two randomly selected training samples and $\alpha \in (0, 1]$ controls the interpolation strength.

Risk assessment: High. Mixup was designed for natural-image classification where interpolated images remain perceptually plausible. For S_{11} traces, an interpolation between a **dry** trace and an **ice** trace produces a spectrum that does not correspond to any physical sensor state. Furthermore, the soft label $\tilde{y}$ is uninterpretable in the sensor context: there is no meaningful “40 % dry, 60 % ice” state.

Recommendation. Mixup should be used *only* for intra-class interpolation (both $\mathbf{x}_a$ and $\mathbf{x}_b$ from the same class) or avoided entirely. If employed inter-class, the benefit must be demonstrated via careful ablation against a non-Mixup baseline, and the interpretability implications must be documented.

7.7 Image-Domain Augmentations for Route B

Route B encodes S_{11} traces as 2D images (e.g., magnitude heatmaps, spectrograms, or recurrence plots). Standard computer-vision augmentations become available, but their applicability must be evaluated against the underlying physics:

- **Horizontal flip** (frequency-axis reversal): reverses the order of frequency bins. This has no physical interpretation — S_{11} at 8 GHz and S_{11} at 12 GHz are not interchangeable — but may act as a form of regularisation if the classifier does not rely on absolute frequency position. *Risk: Medium.*

- **Vertical flip** (magnitude-axis reversal): inverts the magnitude scale, creating physically impossible positive-dB returns. *Risk: High. Not recommended.*
- **Random rotation**: rotates the image by an arbitrary angle. A magnitude-vs. -frequency plot rotated by even a few degrees mixes frequency and magnitude axes, destroying the physical meaning of both. *Risk: Very high. Not recommended.*
- **Random crop / resize**: crops a sub-region and resizes to the original dimensions. Acceptable only if the crop preserves the full frequency range; cropping out a resonant feature removes class-discriminative information. *Risk: Medium.*
- **Colour jitter / brightness perturbation**: for single-channel greyscale images, brightness perturbation is analogous to magnitude scaling (Section 7.3) and shares the same justification and risk level. *Risk: Low.*

7.8 Augmentation Summary Table

Table 8 consolidates all augmentation techniques, their physical justification, risk level, and route compatibility.

Table 8: Summary of augmentation techniques for S_{11} trace data.

Technique	Physical justification	Risk	Route
Additive Gaussian noise	VNA thermal / receiver noise floor	Low	A, B
Smooth freq.-local jitter	Calibration drift between sessions	Low–Med	A, B
Magnitude scaling ($\pm$ dB)	Cable loss / connector variation	Low	A, B
Phase perturbation	Electrical-length / temperature drift	Medium	A
Frequency-axis shift	Variable centre-freq. (conditional)	Med–High	A
Mixup (inter-class)	None (mathematical regulariser)	High	A, B
Mixup (intra-class)	Interpolation within physical state	Medium	A, B
Horizontal flip (image)	None (regulariser)	Medium	B
Vertical flip (image)	None (non-physical)	High	B
Random rotation (image)	None (non-physical)	Very high	B
Random crop / resize	Sub-band analysis (if full range kept)	Medium	B
Brightness perturbation	Analogous to magnitude scaling	Low	B

7.9 Augmentation Budget and Ablation

The number of augmented copies per original sample (the *augmentation factor*) must be chosen with care. A factor of k transforms the effective training-set size from N_{train} to $(1 + k) N_{\text{train}}$, but the information content does not scale linearly because augmented samples are correlated with their parents. Diminishing returns set in rapidly; empirical guidance from the small-sample regime suggests $k \in [2, 5]$ as a reasonable starting range.

Ablation protocol. For every augmentation technique that is included in the final pipeline, a controlled ablation must be performed:

1. Train the model *without* the augmentation and record validation performance (mean $\pm$ std over repeated CV).
2. Train with the augmentation under identical hyper-parameters and compare.
3. Retain the augmentation only if it produces a statistically meaningful improvement (e.g., paired t -test or Wilcoxon signed-rank test on fold-level metrics).

This evidence-based approach prevents the accumulation of augmentations that individually seem harmless but collectively distort the training distribution away from the true data manifold.

References

- Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. ISBN 978-0-262-03561-3. URL <https://www.deeplearningbook.org>.
- Google. TensorFlow lite: Deploy machine learning models on mobile and edge devices. <https://www.tensorflow.org/lite>, 2023. Accessed: 2024-01-15.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.
- Simon Haykin. *Neural Networks and Learning Machines*. Pearson, Upper Saddle River, NJ, 3rd edition, 2009. ISBN 978-0-13-147139-9.
- David W. Hosmer, Stanley Lemeshow, and Rodney X. Sturdivant. *Applied Logistic Regression*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2013. ISBN 978-0-470-58247-3.
- IBIS Open Forum. Touchstone® file format specification, version 2.0. Technical report, IBIS Open Forum, 2009. URL https://ibis.org/touchstone_ver2.0/touchstone_ver2_0.pdf. Defines the standard `.s1p/.s2p/.snp` file format for S-parameter data interchange.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456. PMLR, 2015.
- Keysight Technologies. Reading and writing S-parameter data files with the ENA series network analyzers. Technical Report Application Note 5989-6785EN, Keysight Technologies, 2017. URL <https://www.keysight.com/>. Covers Touchstone v1/v2 import/export and custom ASCII formats.
- Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence (IJCAI)*, volume 2, pages 1137–1143, San Francisco, CA, 1995. Morgan Kaufmann.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>. Originally published as arXiv:1711.05101 in 2017.
- Microsoft. ONNX runtime: Cross-platform, high performance ml inferencing and training accelerator. <https://onnxruntime.ai/>, 2023. Accessed: 2024-01-15.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

- David M. Pozar. *Microwave Engineering*. John Wiley & Sons, Hoboken, NJ, 4th edition, 2011. ISBN 978-0-470-63155-3.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer-Verlag, New York, 1995. ISBN 978-0-387-94559-0.
- Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7(1):91, 2006. doi: 10.1186/1471-2105-7-91.
- Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://openreview.net/forum?id=r1Ddp1-Rb>.